



Sailor: Automating Distributed Training over Dynamic, Heterogeneous, and Geo-distributed Clusters

Foteini Strati*
ETH Zurich
Switzerland

Zhendong Zhang
ETH Zurich
Switzerland

George Manos
ETH Zurich
Switzerland

Ixeia Sánchez
Pérez†

Qinghao Hu
MIT
USA

Tiancheng Chen
ETH Zurich
Switzerland

Berk Buzcu
HES-SO
Switzerland

Song Han
MIT
USA

Pamela Delgado
HES-SO
Switzerland

Ana Klimovic
ETH Zurich
Switzerland

Abstract

The high GPU demand of ML training makes it hard to allocate large homogeneous clusters of high-end GPUs in a single availability zone. Leveraging heterogeneous GPUs available within and across zones can improve throughput at a reasonable cost. However, training ML models on heterogeneous resources introduces significant challenges, such as stragglers and a large search space of possible job configurations. Current systems lack support for efficiently training models on heterogeneous resources. We present Sailor, a system that automates distributed training over heterogeneous, geo-distributed, and dynamically available resources. Sailor combines an efficient search space exploration algorithm, accurate runtime and memory footprint simulation, and a distributed training framework that supports different types of heterogeneity to optimize training throughput and cost.

CCS Concepts: • Computing methodologies → Machine learning.

Keywords: Distributed Training

ACM Reference Format:

Foteini Strati, Zhendong Zhang, George Manos, Ixeia Sánchez Pérez, Qinghao Hu, Tiancheng Chen, Berk Buzcu, Song Han, Pamela Delgado, and Ana Klimovic. 2025. Sailor: Automating Distributed Training over Dynamic, Heterogeneous, and Geo-distributed Clusters. In *ACM SIGOPS 31st Symposium on Operating Systems Principles (SOSP '25)*, October 13–16, 2025, Seoul, Republic of Korea. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3731569.3764839>

1 Introduction

GPUs are in high demand for large-scale Machine Learning (ML). As ML models continue to grow exponentially in

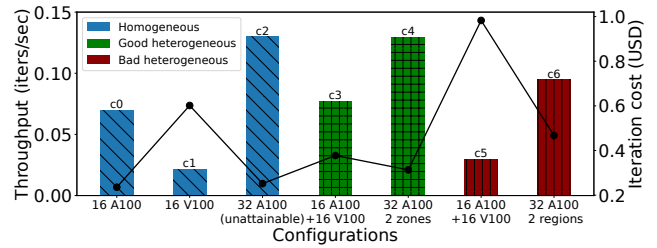


Figure 1. When homogeneous resources are limited, using heterogeneous GPUs (A100, V100) or multiple zones can increase OPT-350M training throughput (c3-c4) at low cost (black line). However, when the resource topology and job parallelization are not well selected, iteration time and monetary cost may increase significantly (c5-c6).

size, they require an increasing number of GPUs to train and fine-tune. This high demand makes it difficult for model developers to allocate the desired number of accelerators to train models at high throughput in public clouds or enterprise clusters [56, 62, 73]. Datacenters typically host a variety of GPU types and generations, spread across geographic regions [10, 22, 23, 66, 70]. Yet model developers tend to restrict model training to *homogeneous* clusters of GPUs, since state-of-the-art distributed training frameworks like Megatron-LM [54] and DeepSpeed [12] assume homogeneous GPUs and inter-node bandwidth. The demand for large, homogeneous GPU clusters compounds the scarcity of high-end GPUs.

Allowing a training job to run on heterogeneous GPU types and/or GPUs distributed across zones (i.e., with heterogeneous inter-node bandwidth) can give model developers access to *more GPUs* per job to increase training throughput. For example, consider a model developer who seeks to maximize training throughput by using 32 A100 GPUs (c2 in Figure 1), but discovers that only 16 A100s (c0 in Figure 1) are currently available in one zone. Using heterogeneous GPU generations (e.g., c3 uses an additional 16 V100s in the zone) or multi-zone configurations (e.g., c4 uses 32 A100s spread across 2 zones within a region) increases throughput by 1.15× and 1.87×, respectively, with a moderate increase in monetary cost per iteration (black line).

*Correspondence to foteini.strati@inf.ethz.ch

†Work done while at ETH Zurich



This work is licensed under a Creative Commons Attribution 4.0 International License.

SOSP '25, October 13–16, 2025, Seoul, Republic of Korea

© 2025 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-1870-0/2025/10.

<https://doi.org/10.1145/3731569.3764839>

However, supporting heterogeneous, geo-distributed resources introduces several challenges. First, heterogeneous GPU types and placements across zones exponentially expand the configuration search space. Searching for an optimal configuration requires jointly optimizing the *resource allocation* and *job parallelization plan* (e.g., data/pipeline/tensor parallelism degrees). For example, in Figure 1, although **c5** uses the same number of GPUs as **c3**, it achieves much lower throughput due to a suboptimal parallelization plan. The search must also consider the cost caused by extra resources and data transfers. Cloud providers charge significant fees for inter-zone and inter-region data transfers [4, 5, 11], which impacts geo-distributed configurations. In Figure 1, **c6** uses the same number of GPUs and parallelization strategy as **c4**, but spreads training across regions (instead of zones within the same region), which increases cost.

Furthermore, as resource availability changes dynamically in datacenters, due to variable demand and node failures or preemptions, it is necessary to navigate this vast search space quickly [60, 63]. Figure 2 shows the varying number of A100 GPUs that we were able to allocate over an 8-hour period, out of the 8 A100s that we continuously requested in 2 different zones in Google Cloud. Such changes in resource availability require frequently re-evaluating the job configuration search space. Thus, model developers require a system to quickly navigate the large search space of heterogeneous (and homogeneous) configurations with dynamic resource availability, optimizing for the user’s performance-cost objective, while satisfying constraints (e.g., budget limits).

Second, profiling many candidate configurations to evaluate their throughput is prohibitively expensive and time-consuming. Hence, it is critical to accurately estimate the iteration time of a candidate configuration. This is challenging in heterogeneous environments, as differences in peak FLOPS, memory capacity, CPU-GPU interconnects, number of GPUs per node, and inter-node bandwidth typically lead to *stragglers*, which can significantly limit throughput. More importantly, variability in memory capacity per GPU may cause *out-of-memory (OOM) errors* in some GPUs, disrupting the entire training job. Simulating iteration time and checking that job configurations are valid (i.e., will not cause OOM) requires correctly modeling stragglers and per-GPU memory footprints.

Finally, after finding an appropriate resource allocation and parallelization plan for a training job given the available resources, model developers need to be able to run this job configuration in a distributed training framework (such as Megatron [54]). We find that optimal configurations for jobs running on heterogeneous resource topologies often include heterogeneous parallelism degrees per stage to load-balance the compute and memory capacity of different GPU types. Today’s state-of-the-art distributed training frameworks need to be adapted to support such heterogeneous job configurations. Furthermore, as resource availability can

change frequently (e.g., when using spot instances [60]), the training framework must be able to quickly reconfigure jobs.

Existing systems do not adequately solve these challenges. First, current works do not co-optimize the resource allocation with the job parallelization plan. Instead, systems like Aceso [31], Galvatron [37], and others in Table 1 expect the user to select a fixed resource allocation for which the system recommends a job parallelization plan. Most systems also do not consider heterogeneous resource topologies. Recent systems like Atlas [41], DTFM [74], Metis [62], and FlashFlex [72] optimize parallelization for heterogeneous GPUs or geo-distributed setups, but they suffer from prohibitively long search times (up to hours for configurations with 10s of GPUs) [62], or suboptimal cost functions [72, 74], making them unsuitable for environments with dynamic resource availability. Second, existing systems rely on inaccurate simulators to estimate the training throughput and memory footprint of candidate configurations. For example, Varuna [3] overlooks significant memory sources (e.g., memory needed by the optimizer, communication, etc) when estimating memory footprint, hence recommending configurations that lead to OOM errors. Finally, state-of-the-art distributed training frameworks like Megatron-LM [54] are slow to reconfigure jobs and do not support heterogeneous job parallelization plans or different microbatch sizes per GPU, which is necessary to maximize throughput and minimize cost in heterogeneous clusters.

To this end, we propose *Sailor*, a system for efficient large-scale training over heterogeneous resources with dynamic availability. *Sailor*¹ consists of three components: a configuration planner, a simulator, and a distributed training framework. The *Sailor* planner navigates the search space of resource allocations and job parallelization plan combinations. It recommends configurations that optimize a user-defined objective (e.g., max throughput or min cost) under constraints (e.g., max budget or min throughput). The planner considers heterogeneous GPU and machine types and geo-distributed setups. The planner uses the simulator to accurately model iteration time and memory footprint for any given configuration. Through a combination of dynamic programming and search space pruning with effective heuristics, the planner finds solutions within seconds even for allocations with 100s of GPUs and varying degrees of heterogeneity. This allows *Sailor* to quickly adapt plans based on resource availability. Finally, the *Sailor* training framework adds support for heterogeneous configurations to execute the planner’s configurations. It also adds support for fault tolerance and elasticity, enabling adaptation to changes in resource availability. Together, these components enable *Sailor* to efficiently automate large-scale training in homogeneous, heterogeneous, and/or dynamic resource environments.

¹Sailor is available at <https://github.com/eth-easl/sailor>

System	Support	Search Time (128 A100)
Piper [59]	3D, X, X, X	<1 sec
AMP [27]	3D, X, ✓, X	14 sec
Varuna [3]	3D, X, X, X	< 1 sec
Oobleck [21]	3D, X, X, X	hours
Metis [62]	3D, X, ✓, X	hours
FlashFlex [72]	3D, ✓, ✓, X	3 sec
Galvtron [37]	3D, X, X, X	10s of sec
Aceso [31]	3D, X, X, X	200 sec
DTFM [74]	2D, ✓, X, ✓	125 sec
Atlas [41]	3D, ✓, X, ✓	100 sec
Sailor	3D, ✓, ✓, ✓	< 1 sec

Table 1. Overview of distributed ML training planners. We omit planners that change ML training semantics [65]. The *Support* column stands for: [degrees of parallelism supported, recommends resource allocation, supports heterogeneous GPU types, supports multi-zone]. Search time assumes a cluster of 128 A100 GPUs and OPT-350M model.

We evaluate Sailor in various setups and compare it extensively to prior works. To the best of our knowledge, our work is the first to compare the major open-source ML training planners proposed to-date (Table 1) in homogeneous and heterogeneous scenarios. We show that Sailor can find resource allocations and job parallelization plans that result in higher throughput than baselines in both homogeneous and heterogeneous clusters. We show how Sailor can leverage heterogeneous resources to improve throughput by 1.1-2.87× compared to the heterogeneity-aware baselines (Metis, FlashFlex, AMP), while maintaining search times of 10s of seconds compared to minutes or hours needed by the baselines. We also show Sailor’s ability to increase performance and reduce cost in geo-distributed setups compared to DTFM by 5.9× and 9.8×, respectively. Finally, we demonstrate Sailor’s ability to minimize monetary cost given throughput constraints, resulting in 40% cost savings compared to the second-best-performing baseline.

2 Background

2.1 ML job parallelization strategies

Million or billion-parameter ML models train on massive datasets on clusters of high-end accelerators such as GPUs, using a combination of parallelization strategies:

Data Parallelism (DP): The model is replicated across workers, while the dataset is partitioned. At the end of each iteration, the workers synchronize their gradients using an all-reduce collective [50].

Pipeline Parallelism (PP): The model is split into stages, with each stage consisting of a set of layers, and assigned to a worker or node. Workers operate at the granularity of a minibatch², performing forward and backward passes, sending activations to the next stage, and gradients to the previous stage. Due to inter-stage dependencies, pipeline

²A minibatch is split in microbatches.

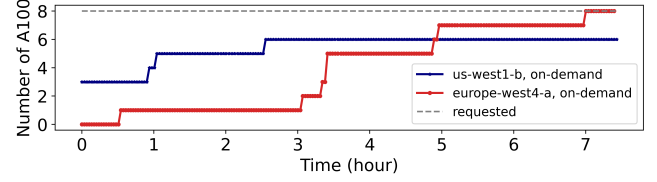


Figure 2. Availability of A100 GPUs in two zones in Google Cloud over 8-hr period. We request 8 GPUs in each zone. The trace was collected in April 2024.

parallelism is subject to *bubbles*, i.e., periods that a stage remains idle, waiting for others to complete [18]. As a result, many approaches have been proposed to process multiple microbatches simultaneously and reduce bubbles [3, 40].

Tensor Parallelism (TP): With tensor parallelism, a layer is divided across GPUs. After each GPU performs its local computations (both in forward and backward pass), the GPUs are synchronized using collectives such as all-reduce and all-gather. Since TP requires frequent communication, it requires very high interconnection bandwidths and is usually limited within a single node for reasonable throughput [54].

2.2 Automating parallelization strategies

Determining the optimal degree of parallelism for each dimension (DP, PP, TP) is complex and greatly affects training throughput. Multiple systems, which we refer to as *planners*, automate this process (see Table 1). Given a fixed resource allocation (e.g., 16 nodes with 4 A100 GPUs each), model configuration (including hyperparameters like global batch size and learning rate), model profiling information (e.g., time for forward and backward pass for different configurations), and hardware characteristics (e.g., network bandwidth), planners explore different parallelization strategies, estimating the training time under different configurations, using some form of simulation. Planners apply techniques such as exhaustive search [3, 62], dynamic programming [27, 59], and integer linear programming [78] to identify configurations that minimize training time.

3 Motivation and Challenges

3.1 Why use heterogeneous, geo-distributed GPUs?

Homogeneous high-end GPU clusters are scarce. The widespread adoption of ML, and hardware vendors’ inability to keep up with this pace, has significantly increased GPU demand. Several studies report limited GPU availability across public cloud providers [8, 9, 56, 60, 73]. For example, Figure 2 plots the availability of on-demand A100 GPUs in two zones in GCP. In one zone, it took 7 hours to allocate 8 A100 GPUs, while in the other zone, the requested number of GPUs was not attained within the 8-hour window. Our findings align with the AWS GPU availability plot from Guo et al. [16], which shows that high-end GPUs such as A100 and H100 are difficult to acquire, while mid-tier GPUs (A10G,

V100, T4) have higher but still limited and dynamic availability. Using GPUs across zones with heterogeneous types gives ML jobs the opportunity to use more GPUs to further increase throughput, as shown in Figure 1.

Power and cooling limits #GPUs per datacenter. From 2020 to 2025, the size of state-of-the-art ML models has increased by roughly $1200\times$ [35, 67, 68], while per-GPU memory capacity has only increased by $8\times$ [69]. This requires hyperscalers to deploy more and more GPUs [17, 61]. However, the high power and cooling requirements of high-end GPUs limits the amount that can be deployed per datacenter [55]. The next generation of ML models may need to train on GPUs across multiple availability zones [6, 30, 42, 52].

Using old GPUs can reduce embodied carbon. Embodied carbon (greenhouse gas emissions associated with manufacturing to disposal) is a major source of datacenter emissions [1, 48, 64]. Although users prefer to allocate the latest GPUs for their ML jobs, older GPUs are abundant as the typical lifetime for ML servers in hyperscaler datacenters is ~ 6 years [51, 64]. Finding optimal ways to spread jobs across heterogeneous GPUs will enable leveraging older GPUs for longer to better amortize embodied carbon.

3.2 Challenges with Heterogeneous ML Clusters

C1: Quickly searching a vast configuration space. Considering heterogeneous and geo-distributed GPUs creates a vast and complex search space. The ML developer needs to decide how many GPUs to use and how to group them across VMs. This complexity increases further, when accounting for job parallelization plans within each allocation. Furthermore, the optimal allocation and partitioning strategies depend on user objectives and constraints. A planner needs to *quickly* navigate the large configuration space to adapt to dynamic resource availability in cloud and on-premise environments [15, 16, 60, 66], since maximizing throughput requires adapting parallelization strategies with changes to cluster topology [3, 63].

Table 1 shows that Metis [62], FlashFlex [72], Cephalo [16], Atlas [41], and DTFM [74] explore parts of this large search space. Atlas [41] and DTFM [74] target geo-distributed training, but do not consider heterogeneous GPU types, and do not decide the various parallelism degrees: instead, they take as input the parallelism degrees, and assign these degrees in the available zones. On the other hand, Metis [62], FlashFlex [72] and Cephalo [16], consider heterogeneous GPU types, but overlook geo-distributed training, and are quite inefficient for dynamic environments. Metis needs a few hours to devise a plan for a 16-GPU cluster (A100 and V100)³, making frequent reevaluation infeasible as GPU availability

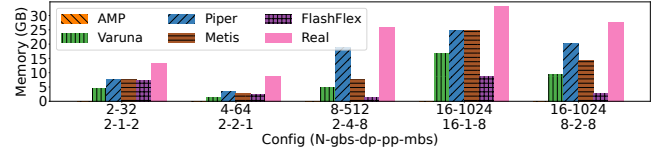


Figure 3. Peak Memory estimations of various baselines compared to the actual peak memory, for the OPT-350M model on a homogeneous cluster of 4 Grace-Hopper per node. N stands for the number of nodes, gbs is the global batch size, pp pipeline parallelism, tp tensor parallelism, dp data parallelism, and mbs the microbatch size.

changes. Cephalo [16] has a search time of 300 sec on a cluster of 64 GPUs⁴, but it is limited only to Fully Sharded Data Parallelism. FlashFlex [72] has a short runtime, but provides inaccurate runtime estimations, leading to suboptimal plans. Furthermore, these planners only optimize for *throughput*, ignoring budget constraints, and cost (dollars per iteration), which affect the optimal configuration (§5.2.4).

C2: Accurately simulating memory footprint and iteration time. Most planners use analytical models or simulations to evaluate a configuration (parallelism degrees, microbatch sizes, etc) on a given cluster setup, since it is impractical and very expensive to deploy and profile every configuration. This evaluation usually includes two stages: 1) memory footprint estimation, to identify whether a configuration is valid (i.e., it does not lead to OOM errors) and 2) iteration time estimation, to determine performance.

Unfortunately, these estimations are often inaccurate, resulting in suboptimal or invalid plans. First, planners either completely ignore memory footprint [27], or underestimate the amount of memory requirements during training [3, 21], omitting activations, optimizer states, and memory fragmentation, or assume the training memory footprint is uniform across all devices and pipeline stages [29, 72]. As a result, these systems may find plans that cause OOM errors, when deployed. Figure 3 compares memory footprint estimations with the real footprint on a homogeneous cluster of up to 16 Grace-Hopper nodes for the OPT-350M model, showing that planners can be 25-95% off when estimating memory footprint. Second, planners often poorly model training time, due to incorrect assumptions about network bandwidth and ignoring communication-computation overlap [12, 76].

Modeling memory footprint and iteration time becomes even more complex with heterogeneity [38]. Different GPU generations vary in compute performance (e.g., TFLOPs), and memory capacity [79]. Thus, a configuration that fits in one GPU, might cause OOM errors in another GPU. Additionally, stragglers and network bandwidth differences (especially in geo-distributed setups [56]) must be considered for accurate timing estimations. Planners must accurately model compute,

³with max_permutation_length and device group variance set to 10 and 0.5 respectively, according to the paper [62]

⁴reported on the paper [16]

memory, and network bandwidth heterogeneity - yet, as shown in Table 1, most systems overlook this.

C3: Supporting both heterogeneous plans and seamless elasticity in a real distributed training framework. Most training frameworks, i.e., systems that train a model on a set of devices, assume homogeneous clusters and job configuration plans. For example, widely used and high-performing frameworks such as Megatron [54] and DeepSpeed [12] assume *uniform* parallelism degrees for the entire training job (e.g. DP=2, PP=6, TP=1). This limits efficiency in heterogeneous configurations, where various parallelism degrees per training subproblem (e.g., using PP=6, with first 3 stages having TP=4, and the next 3 stages having TP=2) help load-balance compute and memory on GPU nodes with different resources. Thus, a framework should accommodate heterogeneous degrees of parallelism. Furthermore, the framework should seamlessly reconfigure and adapt to dynamic GPU availability (Figure 2), as long job reconfiguration times in response to resource changes are wasteful [60, 63]. Although related works propose elastic systems [3, 14, 60] or systems that support heterogeneity [36, 72], there is no open-source system that supports both.

4 Sailor

To address the above challenges, we propose *Sailor*, a distributed training ecosystem that consists of a profiler, planner, simulator, and distributed training framework. As shown in Figure 4, ML developers submit their model training specifications (model, optimizer, global batch size, etc), resource quotas (the maximum number of GPUs for each type and zone), an objective (e.g., maximize throughput or minimize cost), and optionally also constraints (e.g. a maximum budget per iteration or a maximum iteration time). Sailor also receives feedback about the current availability of hardware resources (which may be less than the quotas).

Workflow. The Sailor profiler ① collects information about the training job, the compute nodes and network bandwidth (§4.1). The Sailor planner ② uses this information to select a near-optimal *resource allocation* from the pool of available hardware and a *job parallelization plan* that optimizes the user’s objective within the provided constraints (§4.2). The planner uses the simulator ③ to accurately evaluate various candidate plans (§4.3) in terms of throughput, memory footprint, and cost. Sailor then launches the job with the selected configuration using its distributed training framework ④ (§4.4), which is implemented on top of Megatron-DeepSpeed [12]. Sailor dynamically re-configures the job as resource availability changes.

4.1 Sailor Profiler

Training job profiling: When a user submits a training job, the Sailor profiler collects information about the job’s compute and memory requirements. Sailor profiles a training job

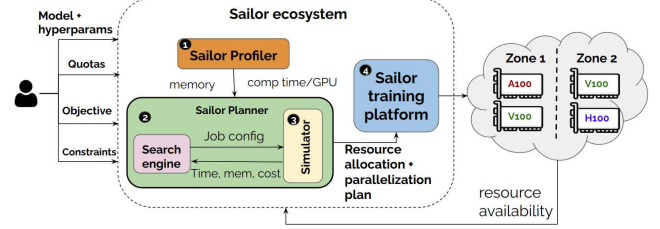


Figure 4. Sailor system overview.

on a *single* GPU node for each different GPU node type in the available resource pool. To minimize profiling time and enable single-node profiling, the profiler reduces repeated layers to a single instance (e.g., it uses one transformer layer for a given LLM). We use PyTorch hooks [44] to collect information per layer: the time required for the forward pass, backward pass, and update phase with different microbatch sizes and tensor parallel degrees. We use CUDA Events for accurate GPU measurements [43]. We also track the number of parameters, output activation, and the memory required for intermediate stages per layer, using the PyTorch CUDA memory allocator [45]. The profiling overhead is negligible (a couple of minutes) for the LLMs we consider. For models with non-uniform layers, all layers need to be profiled, which increases profiling time. Our profiling approach can be used with any dense layers, while profiling Mixture-of-Experts (where layer load may vary dynamically during training) is left for future work.

Cluster profiling: Sailor also collects information about the network bandwidth between any pair of different machine types. Since network bandwidth depends on the message size, Sailor collects network bandwidth measurements (using PyTorch collectives with NCCL backend) by varying the message size and fitting a polynomial function to get a set of coefficients for any pair of node types. Our profiling methodology applies to all GPU types. Adding a new GPU type requires collecting model and cluster profiling data as described above.

4.2 Sailor Planner

The Sailor planner takes as input the training job and cluster profiling information from the profiler, a performance or cost objective, and optionally a constraint such as a budget. The planner selects a resource allocation and a parallelization plan to optimize the objective under the constraints. The parallelization plan defines the number of pipeline stages P (which we refer to simply as *stages*), the data parallelism degree of each stage D , the D pairs of $(GPU_j, TP_j, Zone_j)$ for each stage, and the microbatch size mbs . The Sailor planner does not change the global batch size, thus it does not affect the job’s training dynamics.

The combination of resource allocation and parallelization plan candidates creates a vast search space. To find efficient

solutions *quickly*, Sailor: 1) prunes the search space with heuristics that consider training memory footprint, GPU capacity, and scalability constraints (§4.2.1), and 2) applies dynamic programming to reuse information about the performance of parallelization plans (§4.2.2). These techniques allow Sailor to find training plans in 10s of seconds even for large heterogeneous, geo-distributed scenarios (§5.2).

The planner iterates through different parallelism degrees and microbatch sizes (based on model characteristics and profiling information). For a given layer partitioning and microbatch size, it finds the tensor parallelism degree for each GPU type, based on memory constraints and scaling heuristics. Then, it iterates through all cloud region combinations and selects the data parallelism degrees to evaluate for a combination. For a fixed data parallelism degree, the planner invokes the `solve_dp()` function (Listing 1) that applies dynamic programming to determine the optimal stage configuration (§4.2.2). The planner considers the configuration valid only when it is within the user-specified constraint.

Finally, the planner sorts the configurations according to the objective and returns the best configuration.

4.2.1 Pruning the large search space with heuristics.

We introduce heuristics to prune the search, omitting cases early-on that would lead to suboptimal or invalid results:

H1: Limit tensor parallelism within a node. Tensor parallelism performance is known to degrade when spanning multiple nodes [54, 58]. We restrict tensor parallelism to a single node and do not explore cross-node pairs (unlike Metis [62]). As a result, each tensor parallel replica of a stage only uses a single GPU type.

H2: Prune OOM configurations early. Since each stage replica performs tensor parallelism only among GPUs of the same type, we can easily compute the *minimum* tensor parallelism degree of each GPU type, for a given pipeline parallel stage and microbatch size. We exclude cases with tensor parallelism below this minimum. To find the minimum tensor parallelism degree per GPU for a stage, we compute the memory footprint of that stage as described in §4.3, and identify the minimum number of GPUs required based on the available memory per GPU. The minimum tensor parallelism for each stage is independent of the number of available GPUs per type, so we can reuse it when resource availability changes.

H3: When maximizing throughput, consider data parallelism degrees in decreasing order, until throughput stops increasing. Sailor uses the same data parallelism for each stage. We observe that, with a fixed pipeline parallel degree, increasing data parallelism (by using more machines) benefits training throughput as more pipelines process minibatches independently. However, as the data parallelism degree increases, the time required for gradient synchronization also increases, negatively affecting training throughput. Thus, when optimizing for throughput, we first determine the maximum feasible data parallelism degree based on available

resources, and then progressively reduce it, until throughput stops improving.

H4: When minimizing cost, consider data parallelism degrees in increasing order, until cost per iteration stops decreasing. Following the logic from H3, for a fixed pipeline, doubling the data parallelism degree will lead to doubling the number of resources, but will not half the iteration time (due to all-reduce scaling overheads). Thus, configurations with a lower data parallelism degree lead to lower cost/iteration. Thus, when the user objective is minimizing cost/iteration, we search for *increasing* data parallel degrees D until a solution within the throughput constraint is found.

H5: Keep data parallel communication within a single region, while spreading pipeline parallel communication across more than one region. As shown by earlier work [41, 56], data parallelism performs poorly across regions due to the low network bandwidth. We constraint all data parallel pairs of a stage within a single region.

H6: Treat multiple zones within the same region as a single zone. Within a cloud region, the network bandwidth across zones is similar to the network bandwidth within a zone [56]. Thus, to reduce the search space, we consolidate all zones in a region into a single zone and do the geo-distributed partitioning at a region granularity.

4.2.2 Selecting per-stage configurations with dynamic programming.

We now describe how we find optimal resource configuration per stage for a given pipeline and data parallelism degree, microbatch size, tensor parallel degrees per stage, and GPU type. The formulation we describe below assumes an iteration time minimization objective and §4.2.3 describes how we further incorporate cost constraints. For brevity, we omit the reverse optimization (monetary cost minimization under throughput constraints).

Problem formulation and goal: Given a pipeline parallel degree P , a microbatch size mbs , a data parallel degree D , and tensor parallel degrees tp_{ij} for GPU j and stage i , we want to find, for each stage i , the D replicas, where each replica is a tuple $(j, tp_{ij}, zone_k)$ that minimizes iteration time. Note that Sailor precomputes tp_{ij} (Heuristic H2).

Why dynamic programming? Optimizing resource allocation per stage is challenging, as assigning resources to one stage impacts overall runtime and availability for other stages. In a heterogeneous, multi-zone setup, the number of possible resource combinations per stage explodes. Related works on homogeneous clusters used Integer Linear Programming (ILP) [21, 78], exhaustive search [3], or dynamic programming [27, 59]. We adopt dynamic programming for its ability to decompose the problem into subproblems and reuse intermediate results.

Dynamic programming formulation: Assume we have a pipeline with degree P , and we want to solve the dynamic programming problem for stage i that has L layers, with l_0 being the first layer. We formulate selecting a resource

configuration per stage as finding r resources to give to stage i while minimizing the iteration time T_{iter} as follows:

$$T_{iter}[l_0][P][R] = \min_r \{T_{total}(T_{iter}[l_0+L][P-1][R-r], T_i(r))\} \quad (1)$$

for a given pipeline parallel degree P , stage i , and available resources R . We give r resources to stage i , and $R - r$ resources are available for the subsequent stages. In Sailor, r represents a map of different GPU types and zones, and should be enough to get D replicas of this stage. Based on heuristic $H5$, we keep each stage within a single region. T_{total} is calculated by identifying the straggler between stage i and the rest pipeline, the pipeline communication cost between stage i and $i + 1$, and the synchronization bottleneck between stage i and the rest pipeline.

More specifically, assuming $X - i$ is the pipeline without stage i , and N_b is the number of microbatches processed per pipeline, t_j is the time per stage, and $sync_j$ is the time for synchronization:

$$T_{total} = \max(t_i(r), \text{Straggler}_{X-i}(R-r)) \cdot N_b \\ + \max(sync_i(r), sync_{X-i}(R-r)) + \sum_0^{P-1} t_j$$

Our formulation aims to minimize iteration time and reduce pipeline stragglers, inherently excluding imbalanced pipelines with large stage time differences.

Implementation: Listing 1 implements the above formulation for heterogeneous GPUs and various cloud zones and regions. The procedure starts by generating all resource combinations of different GPU types that could fit this stage with the specified data parallelism (line 2). Then, for each combination r , it finds the next available region that fits stage i with r resources (lines 6 and 14). If i is the last stage, it returns the configuration that minimizes the stage time.

4.2.3 Adding a monetary cost constraint. So far, we have formulated the iteration time minimization as a dynamic programming problem, and we split the problem at per-stage subproblems as shown in Eq 1 and Listing 1. When introducing a budget constraint, we need to account for the *remaining* budget per stage, to solve the DP subproblem for that stage. The monetary cost depends both on the resources used and the iteration time. However, the iteration time depends on the pipeline straggler, which is not yet known when solving the resource allocation subproblem for stage i . To overcome this limitation, when solving the subproblem for stage i , we approximate the remaining budget by first considering that stage i is the straggler, and solving for the remaining stages with the respective remaining budget. At the end, we determine the actual straggler. If our straggler assumption was not correct, we adjust the budget with the new straggler and we solve the subproblems again.

Problem formulation: The monetary cost per iteration includes the cost due to compute resources C_{comp} and due

to data transfer C_{comm} . When introducing a monetary cost constraint C , the cost per iteration should satisfy:

$$C_{iter} = C_{comp} + C_{comm} \leq C \Rightarrow \sum_{i=0}^{P-1} C_{comp_i} \cdot T_{iter} + C_{comm} \leq C$$

$$\sum_0^{P-1} C_{comp_i} \cdot \left(\sum_0^{P-1} t_i + N_b \cdot t_{straggler} + \max_0^P(t_{sync_i}) \right) + C_{comm} \leq C$$

where $\sum_0^{P-1} t_i$ stands for the pipeline warmup and cooldown phase, N_b is the number of microbatches processed per pipeline, and t_{sync_i} is the time required for the synchronization of all replicas of stage i . Since large models usually train with large global batch sizes, the $N_b \cdot t_{straggler}$ term usually determines the iteration time, we can rewrite as:

$$\sum_0^{P-1} C_{comp_i} \cdot (N_b \cdot t_{straggler}) + C_{comm} \leq C \quad (2)$$

From a dynamic programming perspective, assuming we are at pipeline stage i , which has a maximum budget limit C_{cur} , the cost constraint will be: $C_i + C_{rem} \leq C_{cur}$, where C_{rem} is the cost of the remaining stages, and C_i is the cost of stage i . From Eq. 2, we have that $C_i = C_{comp_i} \cdot (N_b \cdot t_{straggler}) + C_{comm_i}$. In Listing 1, line 14, when exploring a resource combination r , we can easily find C_{comp_i} , and C_{comm_i} for stage i . Since we do not know $t_{straggler}$, which is required to specify the remaining budget for the next stages, we use the approximation in lines 17-32: We begin assuming stage i is the straggler ($t_{straggler} == t_i$), and compute the C_{rem} for the next stages accordingly. We call the *solve_dp* function for the next stages giving C_{rem} as the budget constraint (line 20). If a solution cannot be found, we proceed with the next resource combination for stage i (line 22). If a solution is found, we check whether it is within the cost limit and keep the one with the maximum throughput (line 26). We also check the straggler of the found solution: if it is the same with the one we assumed, we break, and proceed to the next resource combination (lines 26-28). Otherwise, we adjust the budget with the new straggler (lines 31-32) and iterate again.

4.3 Sailor Simulator

The planner uses the simulator to evaluate the performance and memory footprint of the generated plans. The Sailor simulator takes as input a training job specification (model, global batch size, optimizer, hyperparameters) and a job parallelization plan. It then estimates the memory footprint per GPU, the iteration time, and the cost per iteration. The simulator allows the planner to easily specify different types of heterogeneity: hardware heterogeneity (GPU type, number of GPUs per node, and network bandwidth) and job configuration heterogeneity (number of pipelines and different stage configurations per pipeline). The training configuration also specifies the microbatch size. The simulator also incorporates the information collected by the profiler about the training job and network bandwidth of the used links.

Listing 1. Resource-Stage assignment algorithm

```

1 def solve_dp(P, i, R, D, tp_gpu_stages, Ci):
2     Rcombos = generate_combos(R,D,tp_gpu_stages[i])
3     if i==P-1:
4         time_i = None
5         for r in Rcombos:
6             next_region = find_region_fits(i, r, curr_region)
7             time_ir = time_for_stage(i,r,next_region)
8             cost_ir = cost_for_stage(i, r, next_region, time_ir)
9             if cost_ir <= Ci: time_i = min(time_i, time_ir)
10        T_iter[P][i][R] = time_i
11    else:
12        time_all = None, min_time = None
13        for r in Rcombos:
14            next_region = find_region_fits(i, r, curr_region)
15            time_i = time_for_stage(i,r,next_region)
16            cost_i = cost_for_stage(i, r, next_region, time_i)
17            C_rem = Ci - cost_i
18            assumed_straggler = time_i
19            while (C_rem > 0):
20                nextconf = solve_dp(P, i+1, R-r,
21                next_region, all_regions, C_rem)
22                if nextconf is None: break
23                time_all = total_time(time_i, nextconf.time)
24                cost_all = total_cost(cost_i, nextconf.cost)
25                if nextconf.straggler < assumed_straggler:
26                    if (cost_all <= Ci):
27                        min_time = min(time_all, min_time)
28                    break
29                cost_i = cost_for_stage(i, r, next_region,
30                straggler=nextconf.straggler)
31                C_rem = Ci - cost_i
32                assumed_straggler = nextconf.straggler
33        T_iter[P][i][R] = min_time
34    return T_iter[P][i][R]

```

Both the Sailor planner and simulator treat GPUs as black-box compute units, thus they can seamlessly support GPUs from different generations, vendors, and even different accelerators (e.g. TPUs).

Memory footprint estimation: The Sailor simulator accurately estimates a training job’s memory footprint by: 1) calculating memory footprint per GPU, per-stage and 2) considering *all* main sources of memory footprint during training. Compared to prior works that assume a homogeneous memory footprint per stage, we observe that for a given parallelism configuration, the memory footprint of a training worker depends on its layer partitioning, pipeline stage index, tensor parallelism degree, and microbatch size. Hence, memory footprint varies among workers and needs to be analyzed per worker to detect OOM scenarios.

Second, the peak memory footprint of a worker throughout training consists of various sources, often ignored in prior works. The peak memory footprint M_{peak} of a worker

is given by: $M_{peak} = M_{model} + M_{activation}$, where M_{model} corresponds to the memory needed to keep copies of model parameters and is given by $M_{model} = num_params \cdot mul_factor \cdot data_type_size$. num_params is found by the stage id and tensor parallelism degree of the worker, while mul_factor accounts for multiple copies needed for the model itself, the optimizer, gradients, and communication [46]. $M_{activation}$ is the memory needed for storing layer activations and depends on the stage id and tensor parallelism degree of a worker, and microbatch size. By computing the memory footprint for each worker and comparing with the worker’s GPU capacity, the simulator can easily detect OOM cases.

Iteration time estimation: We define one iteration as a full pass over the user-defined global batch size. The iteration time is calculated as: $T_{iter} = \max(T_{ppi}) + T_{sync} + T_{update}$, where T_{ppi} is the time needed for pipeline i , T_{sync} is the time needed for gradient synchronization at the end of an iteration, and T_{update} is the time for model update. We compute T_{ppi} and T_{sync} following the formulas described in [56] for 1F1B pipeline parallelism, using our profiling for network bandwidth with respect to the message size per network link, to estimate communication time (for peer-to-peer and collectives). For each pipeline, the 1F1B pipeline parallelism schedule includes a warm-up, steady, and cool-down phase per iteration, where the steady phase is determined by the stage with the largest computation time (straggler). After computing the iteration time per pipeline, we compute synchronization time. Taking the maximum time per pipeline accounts for straggler effects caused by heterogeneity in GPU generations, inter-GPU, and CPU-GPU interconnects.

Iteration cost estimation: Related works (Table 1) do not compute the monetary cost of different resource allocation and job configuration combinations, since they only optimize for throughput. However, a very important metric to account for is *cost per iteration*, especially with geo-distributed training, due to costs associated with across-zone communication. Since Sailor does not change the global batch size and training hyperparameters, the number of iterations needed to reach convergence is constant, regardless of the cluster setup and parallelization strategy. Thus, the monetary cost per iteration indicates the total budget needed for the whole training. The metric depends both on the cost of allocated resources, as well as the training throughput and communication cost. The cost per iteration is given by $C_{iter} = C_{comp} + C_{comm}$, where C_{comp} is the cost due to compute resources, and is calculated as $\sum_i (N_i \cdot cost_per_gpu_i) \cdot T_{iter}$, for all different GPU types i in the cluster. C_{comm} stands for the cost for data exchange per iteration (e.g., when using geo-distributed training in public cloud) and is given by $C_{comm} = \sum_{ij} (bytes_{ij} \cdot cost_per_byte_{ij})$ for all zones i, j in the cluster. $bytes_{ij}$ might include traffic for data and pipeline parallel communication.

4.4 Sailor Distributed Training Framework

The Sailor training framework receives the job configuration that the planner generates, sets up the cluster, and starts training. We modified Megatron-DeepSpeed [12] to support heterogeneous plans and seamless elasticity, to make it compatible to planner’s output and allow for fast reconfiguration when resource availability changes.

Support for heterogeneous plans: We added support for varying tensor-parallel degrees across data-parallel pairs per pipeline stage. The framework takes as input a rank topology for each stage, allowing each rank to belong to distinct tensor-parallel groups. Different tensor parallelism per stage affects the pipeline and data parallel communication, requiring workers to split or replicate activations and gradients across multiple peers. To accommodate this, we adjust the PyTorch communication groups and modify the send/rcv and all-reduce operations accordingly.

Support for fault-tolerance and elasticity: The Megatron-DeepSpeed framework lacks failure recovery and dynamic resource reconfiguration: the whole training needs to stop, and the user needs to *manually* reconfigure and restart the job. However, resource availability frequently changes both in the cloud (especially with spot instances) [56, 60] and in on-premise datacenters as jobs start or finish [63]. We introduce modifications for fast reconfiguration. Each job consists of a controller and multiple workers. The workers handle training, while the controller monitors their status and detects resource availability changes. Upon detecting a change, the controller reinvokes the planner to generate a new plan and instructs workers to adjust accordingly. We follow a kill-free approach to minimize reconfiguration time: existing workers destroy the current communication group, clean up their GPU memory, repartition the model, and setup a new communication group. If additional resources become available, the controller waits for new workers to initialize before updating the training configuration. Training restarts from the latest available checkpoint. We use asynchronous checkpointing to minimize the rollback time [39, 57].

5 Evaluation

We evaluate Sailor to answer the following questions:

1. How accurately does the Sailor simulator estimate iteration time and memory footprint?
2. How well does the Sailor planner perform compared to baselines in homogeneous and heterogeneous setups, in terms of throughput and monetary cost?
3. How is the Sailor planner search time affected by the cluster size, resource heterogeneity, user constraints and the different optimizations?

System configurations: We evaluate Sailor using real hardware and simulations. We use different cluster setups

and GPU generations, in both cloud environments and on-premise clusters. In the public cloud, we used VMs with A100-40GB and V100-16GB from GCP [10]. For our on-premise datacenter experiments, we used a cluster with up to 32 homogeneous machines with 4 Grace-Hopper GPUs each, and a cluster of heterogeneous machines with 2x8 Titan-RTX, 3x8 RTX-2080, and 2x8 RTX-3090.

Models: We use the OPT-350M [20] and GPT-Neo-2.7B [19] models, with global batch size of 2048 sequences, and sequence length of 2048 tokens, with the Adam optimizer.

Baselines: To our knowledge, we present the first comprehensive comparison of major planners for large-scale ML training, including planners targeting *homogeneous* resources (Piper [59], Varuna [3], Aceso [31]), *heterogeneous* resources (AMP [27], Metis [62], FlashFlex [72]), and *geo-distributed training* (DTFM[74]). All baselines, except Aceso, are integrated into our platform with a unified Python API. We profile our models once and give each baseline its required profiling information. Aceso defines its own operators, which we profile separately. As Aceso also uses the Megatron backend, per-layer runtime profiles are very close to our models. We used the open-source version of all baselines. DTFM [74] does not determine parallelization strategies (e.g., DP, PP), but instead partitions a given plan. Therefore, we exhaustively generated all homogeneous parallelization plans and applied their partitioning methods to each. As the Atlas paper does not have an open-source implementation of runtime and memory simulation, we were unable to test Atlas end-to-end. However, we tested the zone assignment described in the paper [41], which performs similar to Sailor.

5.1 Validation of the Sailor simulator

We evaluate the accuracy of the Sailor simulator’s iteration time and memory footprint estimations. We vary the number of devices and parallelization plans, find the difference in iteration time and peak memory footprint, and summarize using box plots. We omit AMP and DTFM since they do not support memory estimation.

Cluster of homogeneous GPU types. Figures 5a and 5b show the iteration time and peak memory footprint estimation error for the homogeneous cluster of Grace-Hopper for the OPT-350M model, respectively. Most baselines fail to accurately capture the peak memory footprint, since they ignore significant memory sources and assume a homogeneous memory footprint across the different pipeline stages. The examined baselines exhibit an error of 12.5-74% on average, while Sailor achieves an average error of 5.56%. Sailor also reduces the average runtime estimation error in the homogeneous setup to 6%, compared to 10-20% for the baselines.

Cluster of Heterogeneous GPU types. Figure 6 shows the iteration time prediction error for the OPT-350M model in a heterogeneous cluster of Titan-RTX, RTX-2080, and RTX-3090. The homogeneous planners (Piper, Varuna, Aceso) do not consider the differences in forward and backward passes

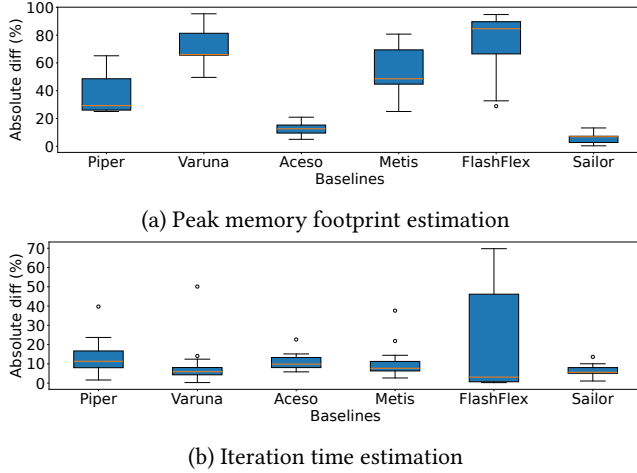


Figure 5. Different planners’ compute and memory estimation on a cluster of GH200 GPU for the OPT-350M model.

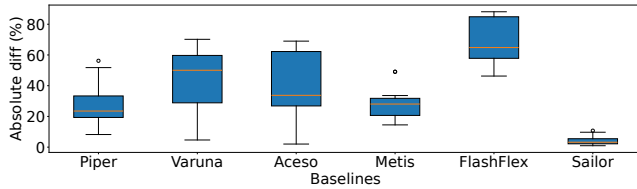


Figure 6. Different planners’ iteration time on a cluster with heterogeneous GPUs for the OPT-350M model.

of the different GPU types, resulting in an average error of 28%, 47%, and 37%, respectively. Even heterogeneous planners fail to accurately capture runtime: since FlashFlex relies on the theoretical performance of GPUs, it cannot accurately predict the runtime, getting an error of 69%. Metis fails to fully capture the heterogeneous network bandwidth between nodes, thus miscalculating the communication cost, resulting in 28% error in iteration time estimation, on average. On average, Sailor’s iteration time estimations error is 4.5%.

5.2 Sailor Planner vs. Baselines

We evaluate the throughput achieved by Sailor and the baselines across various cluster configurations using both real hardware and our simulator. All baselines require a predefined resource topology as input: we consider 4-GPU VMs for each GPU type. Sailor takes resource quotas as input (total number of GPUs per type per zone) and jointly determines both the topology (VM allocation) and the parallelization plan. For Metis, we impose a 300-second time limit and use the best solution found within that period, if available. We summarize key takeaways.

5.2.1 Homogeneous Setups. Figure 7 shows the throughput achieved using the baseline planners and Sailor with only A100 GPUs for the OPT-350M model. Varuna failed to generate a valid plan that would not lead to OOM errors due

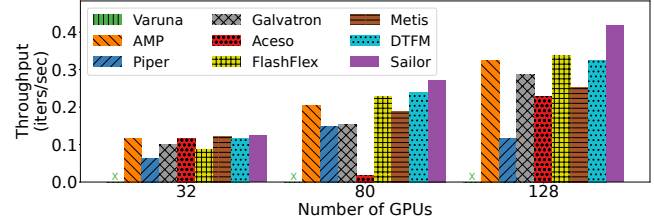


Figure 7. Comparison of the different planners considering A100-40GB GPUs for the OPT-350M model in one zone

to the poor memory estimation, and its limited search space (only supporting 2D parallelism). Sailor improves throughput by 1.15× compared to the closest baseline (DTFM), and even up to 5.7× (compared to Aceso).

5.2.2 Heterogeneous Setups. We evaluate Sailor’s throughput compared to baselines (AMP, Metis, and FlashFlex) in a heterogeneous cluster setup using a mixture of A100 and V100. We vary the ratio and amount of A100 and V100 (50%-50% and 25%-75%) to assess the impact of having more low-end GPUs in the cluster [16]). As in the homogeneous setup, all baselines get a fixed resource topology (4-GPU VMs) as input, while Sailor also determines the resource topology along with the parallelization plan.

Throughput of heterogeneous baselines: Figures 8 and 9 use our simulator to evaluate Sailor and the baselines in large heterogeneous clusters for the OPT-350M and GPT-Neo-2.7B model, respectively. We also compare the throughput achieved by Sailor, when using only homogeneous resources (either A100 or V100). We also report the monetary cost per iteration at each scenario (number on top of bar), and the number of plans generated by the baseline that would lead to OOM before a valid plan was found (**bold** number on top of bar). Although AMP achieves high throughput in the homogeneous case, it performs poorly in the heterogeneous scenario, as it only allows for homogeneous plans, and does not correctly model stragglers. As a result, the monetary cost per iteration also increases. Furthermore, since AMP does not model training memory footprint, it leads to a large number of OOM plans, especially in the case of the GPT-Neo model (Figure 9).

FlashFlex achieves similar or higher throughput than AMP, as it can consider heterogeneous plans, and captures differences in compute between the different GPUs. However, its throughput is still low, as it uses low tensor parallelism and microbatch sizes. This leads to higher iteration costs (e.g. Figure 8), since it uses a large number of resources with a low throughput. It also fails to find valid plans for the large GPT-Neo model due to suboptimal memory estimation. Metis capped at 300 seconds achieves higher throughput than FlashFlex and AMP, due to more accurate runtime and memory footprint estimation, layer partitioning, load balancing, and exhaustive search of different GPU group combinations, but it generates a huge number of OOM plans (Figure 9).

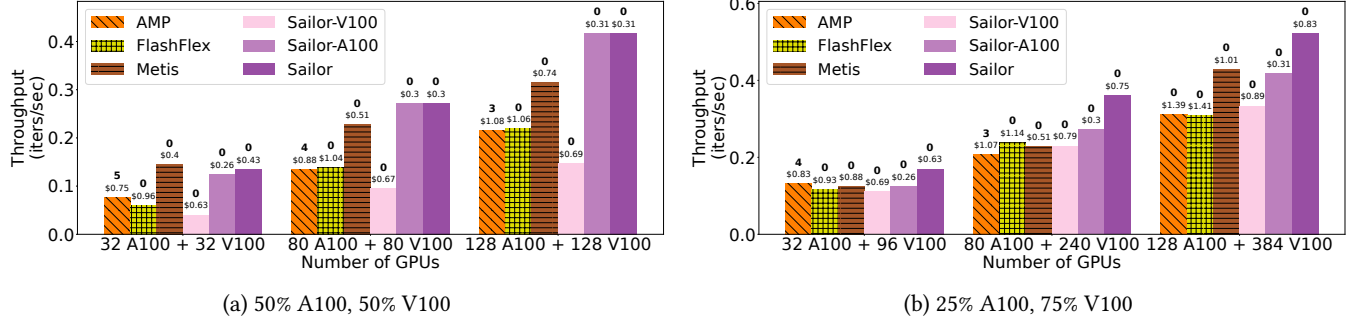


Figure 8. Comparison of planners considering A100 and V100 GPUs for the OPT-350M model in 1 zone

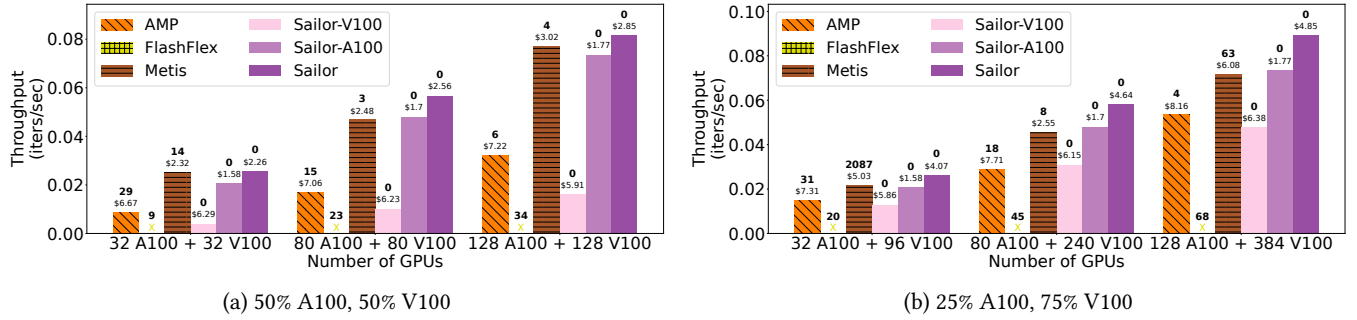


Figure 9. Comparison of planners considering A100 and V100 GPUs for the GPT-Neo-2.7B model in 1 zone

Sailor achieves significantly higher throughput compared to baselines: for the OPT-350M model, Sailor achieves 1.9 $\times$, 2.03 $\times$, and 1.15 $\times$ higher throughput compared to AMP, FlashFlex, and Metis, respectively, when the ratio of A100 and V100 GPUs is equal, and 1.57 $\times$, 1.55 $\times$, 1.39 $\times$ higher throughput when more V100 are available. The speedups are similar for the GPT-Neo model as well. Compared to baselines, Sailor uses larger tensor parallelism and longer pipelines, accounting for data parallelism limitations among heterogeneous nodes (for example AMP uses data parallelism of 256 in Figure 12 which significantly increases the data parallelism communication cost.) Also, Sailor does not output invalid plans, significantly improving the plan deployment compared to baselines with 10s of invalid plans. Sailor’s ability to discover efficient resource topologies and parallelization plans also translates to significantly lower cost compared to the baselines (up to 2.67 $\times$ lower cost compared to baselines in Figures 8 and 9).

We also evaluated Sailor and the other heterogeneous baselines with real hardware using a smaller cluster of A100 and V100 GPUs. Figure 10 shows the throughput achieved for the OPT-350M model when using an equal amount of GPUs per type (8 each), and when using more V100 than A100, as V100 were more readily available. When using the same number of GPUs per type, Sailor outperforms the baselines by 1.08–1.81 $\times$ and does not generate any Out-Of-Memory plans. In contrast, AMP and Metis generate 5 and 1 invalid (OOM)

plans before finding a valid plan, respectively. The valid plan found by Metis is similar to Sailor’s, but Sailor avoids invalid outputs entirely, and finds a solution in less than 1 sec, while Metis takes 60 sec. When using 8 A100 and 16 V100 GPUs, Metis fails to output a plan as it requires the global batch size to be equally divisible by the total number of GPUs. We therefore reuse the plan from the 16 GPU case. AMP produces the same plan for the 24 GPU case as the 16 GPU case, while the plan provided by FlashFlex utilizes all 24 GPUs but uses an unnecessarily large number of pipeline stages that degrades throughput. Sailor outperforms the baselines by 1.19–2 $\times$ for the 24 GPU scenario. Our simulator’s estimated iteration times were within 4% of those measured on real hardware.

Search times: Table 2 shows the search time for Figure 9b for the different baselines. Metis has long search times. In our experiments, we let Metis search for up to 300 seconds and take the best plan it produces in this time window. The other baselines are significantly faster, finishing their search in less than 200 seconds in all cases. Sailor keeps the search time within 1 minute even for the largest case (512 GPUs) due to its efficient search algorithm.

Benefits of heterogeneity: Figures 8, 9 and 10 also show the throughput achieved by Sailor when given homogeneous-only setups of A100 (Sailor-A100) or V100 GPUs (Sailor-V100). Given that V100s are less efficient than A100s, using A100 only, or a mixture of A100 and V100 is always better

Baseline	A100-V100		
	32-96	80-240	128-384
AMP	31.22	51.43	86.41
FlashFlex	4.05	54.67	222.64
Metis	300	300	300
Sailor	1.6	7.67	17.4

Table 2. Search times (in seconds) for Figure 9b.

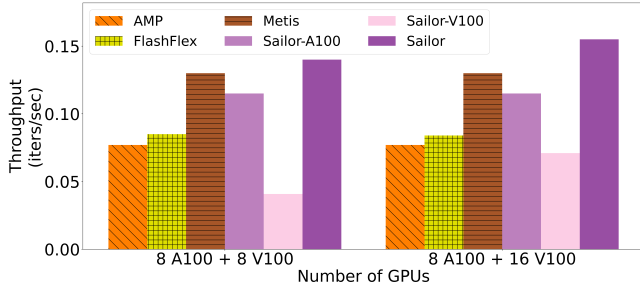


Figure 10. Throughput of heterogeneous planners for clusters of A100 and V100 GPUs for the OPT-350M model.

than using V100 only. However, using V100 *in addition* to A100 does not always improve throughput. Heterogeneity is more beneficial when resources are limited (e.g. Figure 8a, with 32 GPUs per type), or with larger models like GPT-Neo. In fact, for the OPT-350M model, when 128 A100 and 128 V100 are available, Sailor chooses a plan with 128 A100 only, as it determines that no additional benefits will be gained by adding extra resources. Moreover, when the ratio of V100 to A100 is higher than 1, the throughput improvements are more significant, as shown in Figures 10, 8b and 9b, where the V100:A100 ratio is 2:1 or 3:1. This aligns with the GPU memory capacity ratio, as well as the time for forward/backward pass for the transformer layers of the two GPU types, enabling better load balancing. Finally, heterogeneity leads to a higher cost, as more resources are used. Since the objective is to maximize throughput, Sailor ignored monetary cost when searching job configurations. In §5.2.4, we will show Sailor’s ability to consider budget optimization and constraints.

Key takeaway 1: Heterogeneity is most beneficial when resources are limited, or for larger models, or when the ratio of the different GPU types aligns with their memory and compute characteristics for better load balancing.

5.2.3 Geo-distributed setups. In Figures 11 and 12, we evaluate Sailor’s throughput in geo-distributed setups, considering A100 GPUs for the OPT-350M model. We compare Sailor with DTFM, with the exhaustive search to automatically discover parallelization plans. We report training throughput and monetary cost per iteration, taking both the computation and communication cost into account.

Small-scale results on real hardware: Figure 11 shows an experiment with small-scale cluster in 4 cloud zones (2 regions) using 4 and 8 A100 GPUs per zone. DTFM cannot fully

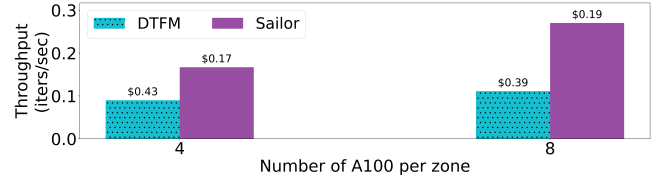


Figure 11. Throughput of geo-distributed planners with A100-40GB for the OPT-350M model in 4 zones (2 regions) using real GPUs.

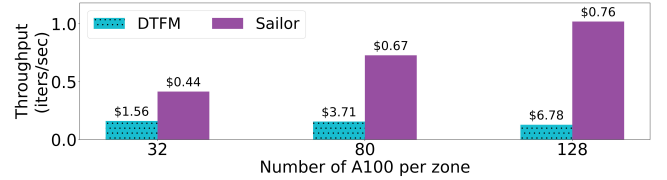


Figure 12. Throughput of geo-distributed planners with A100-40GB for the OPT-350M model in 5 zones (2 regions) using our simulator.

leverage multiple zones, mainly due to its suboptimal cost function and lack of memory footprint estimation. DTFM ranks solutions based on the time spent in data and pipeline parallel communication, which leads to suboptimal solution ranking. Furthermore, it uses all cloud regions, which increases communication bottlenecks and cost without increasing throughput. In contrast, Sailor uses only 1 region with all available zones (us-central1), as incorporating an additional region (us-west1) does not improve throughput. Sailor leads to 1.9× and 2.45× higher throughput than DTFM for two cluster sizes. Our simulator was within 3.7% of the real throughput in this scenario.

Large-scale results on simulation: In Figure 12 we use our simulator to evaluate larger cluster sizes with 5 cloud zones (2 regions). Sailor achieves 5.9× higher throughput and 9.48× lower cost per iteration than DTFM. Sailor employs larger microbatch sizes and tensor parallelism degrees, reducing the pipeline and data parallel data transfers. Furthermore, Sailor finds configurations within 1 second, compared to DTFM that needs hundred of seconds with large clusters (due to the exhaustive search). On GPT-Neo, DTFM fails due to OOMs, while Sailor finds valid plans with a throughput of 0.07–0.21 iters/sec across cluster sizes.

Finally, comparing Sailor’s throughput for the OPT-350M model in the heterogeneous setups with more V100 (Figure 8b) and the geo-distributed A100-only setup (Figure 12), shows that the geo-distributed setup achieves up to 2× higher throughput, and also lower cost.

Key takeaway 2: Efficiently using the same GPU type across zones can lead to higher throughput and lower cost than mixing GPU types within a single zone, despite data transfer costs in geo-distributed setups.

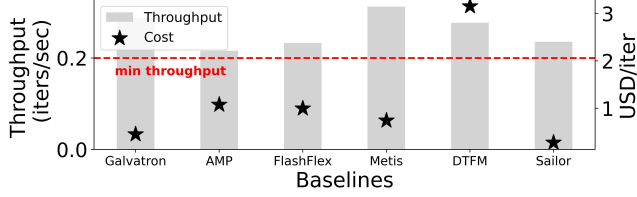


Figure 13. Minimizing cost with a throughput constraint.

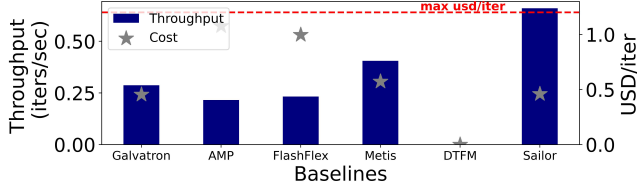


Figure 14. Maximizing throughput with budget limit.

5.2.4 Optimization with constraints. We now change the optimization objective to minimizing monetary cost and add constraints. Since baselines do not support cost-aware optimization or constraints, we modify them to rank solutions by iteration cost and only return plans within the constraints. We consider 2 cloud zones in the same region, each with 128 A100 and 128 V100. Sailor takes the full search space and outputs the resource allocation and parallelization plans. The baselines (that require a fixed topology) assume 4-GPU VMs: Varuna, Aceso, Galvatron consider only the A100 machines (since they are more high-end than V100). AMP, FlashFlex, and Metis consider both A100 and V100 in a single zone, while DTFM considers only A100 in two zones.

Scenario 1: Minimizing cost with throughput constraint of 0.2 iterations/sec: Figure 13 shows the throughput (bars) and iteration cost (asterisks) achieved by the different planners. Sailor outputs a solution within the constraint, while achieving the minimum cost compared to baselines: 40% lower cost compared to the second-best performing baseline (Galvatron). The found solution consisted of 64 A100 GPUs in a single zone, as they were enough to meet the throughput target.

Scenario 2: Maximizing throughput with cost constraint of 1.2 USD/iteration: Figure 14 shows the throughput (bars) and iteration cost (asterisks). Most baselines will use all available resources (e.g. all 128 A100 and 128 V100), even if they do not benefit throughput. DTFM does not find a solution as it outputs plans with low throughput and high costs. Sailor outperforms all baselines leading to 1.65-3× higher throughput while remaining within the cost constraint. Sailor’s plan includes 256 A100 GPUs in two zones, with tensor parallelism of 4, and data parallelism of 64.

GPU types	Search Time		
	Dyn Prog	+ Heuristics	+ cost limit
1	hours	0.25 sec	0.4 sec
2	hours	5 sec	20 sec

Table 3. Breakdown of search time, in dynamic programming and heuristics, and additional search time overhead due to budget constraints for the GPT-Neo-2.7 model. We use A100 and V100 in one zone, with 128 GPUs per type. The budget constraint is 1.5 USD/iteration.

5.3 Sailor scalability study

We evaluated Sailor’s search time varying the number of GPUs and zones or regions with a homogeneous GPU type. The search time remains below 1.5 sec even with 5 GCP zones and 256 A100/zone for the GPT-Neo-2.7B model. In contrast, adding more GPU types has a much higher impact in Sailor’s search time: considering 256 GPUs/type in a single zone, Sailor’s search time is 0.3, 6.2, 4900 sec for 1, 2, 3 GPU types, respectively. Nevertheless, Sailor’s search process is much more efficient than the rest of the heterogeneous baselines: Metis [62] needs hours to complete its search even with 2 GPU types, while FlashFlex [72] cannot find a valid configuration for any of these setups.

5.4 Sailor Planner optimization breakdown

Table 3 shows Sailor’s planner search time breakdown when optimizing for throughput, and budget constraint overhead, considering 128 A100 and 128 V100 GPUs for the GPT-Neo-2.7b model. The dynamic-programming-only approach needs hours to complete the search, even in the single-GPU-type case. Introducing the heuristics *H1-H3* (4.2.1), which apply in this scenario, dramatically decreases the search time to a couple of seconds. The additional cost constraint increases the search time (4× increase in the 2-GPU-type scenario) due to the extra iterations caused by the straggler approximations, as described in 4.2.3.

5.5 Reconfiguration overheads

We measure Sailor’s reconfiguration time on a cluster of 16 V100 GPUs for the OPT-350M model. When 4 more GPUs are added, the controller re-invokes the planner, instructs workers to clean up (e.g., destroy NCCL groups), and broadcasts the new plan and topology. Planning takes 0.1 seconds, process cleanup takes 3 seconds, and topology broadcast (using grpc) takes 1.25 seconds. After the workers have received the new plan and topology, they reinitialize NCCL communication groups (for data, pipeline, tensor parallelism) in 4.5 seconds, redefine the model and optimizer in 2 seconds, redefine the dataloaders in 0.5 seconds, and resume training. While moderate at this scale, NCCL initialization can take minutes on thousands of GPUs [25]. Existing methods to reduce this overhead [25] can be integrated into Sailor.

6 Discussion

Planner and simulator limitations: Our planner and simulator currently support only the 1F1B pipeline parallel schedule, and do not incorporate optimizations such as activation offloading [47] or rematerialization [75]. Adding support for these optimizations is left for future work.

Additional challenges with heterogeneous hardware: Training over heterogeneous and geo-distributed datacenters can introduce additional challenges. Heterogeneity in accelerator vendors (e.g., NVIDIA vs. AMD) and network links (e.g., Infiniband vs. Ethernet) may prevent the use of high-performance collective communication libraries (e.g., NCCL), which often assume uniform network protocols. To maximize performance, collectives must be adapted to heterogeneous links. Furthermore, geo-distributed networks are prone to unreliability, including unpredictable jitter and packet loss. Training frameworks and algorithms should detect and adapt to such issues. Finally, even though Sailor optimizes parallelization strategies, it is based on strategies that were first introduced for homogeneous settings (e.g. 1F1B pipeline schedule). Achieving high performance in such contexts may require developing new schedules that more effectively overlap computation and communication, thereby reducing bubble times [7].

7 Other Related Work

Asynchronous geo-distributed training: Several systems propose training over geo-distributed, preemptible, and heterogeneous resources by introducing asynchrony and reducing communication via quantization and sparsification. DiLoCo [13] uses federated averaging to reduce communication, performing more local computation before synchronization. SWARM [49] proposes a decentralized, model-parallel approach to deal with poorly connected and unreliable devices. CocktailSGD [65] uses gradient compression to improve communication over low-bandwidth networks. These approaches influence training dynamics and are orthogonal to our work. Sailor employs synchronous training, which is preferred in large-scale training [63].

Automatic VM selection: CherryPick [2], RAMBO [28], and PARIS [71] apply Bayesian Optimization or performance modeling to recommend optimal VM types. SkyPilot [73] picks cost-efficient VMs across providers based on workload and user constraints. These methods treat workloads as black boxes and are not tailored for ML training. Siftly [32], Cynthia [77], SpotDNN [53], DeepSpot [26] find optimal configurations of on-demand and spot instances for ML jobs. However, they target small data-parallel jobs, and are inadequate for large-scale training that involves various types of communication, which is our focus.

Heterogeneous and Geo-distributed ML Inference: Recent works have proposed systems for ML inference on

heterogeneous and geo-distributed resources [24, 33, 34]. Although training and inference workloads differ significantly (e.g. in runtime and memory footprint estimation), synchronization and communication patterns as well as planning decisions such as partitioning models across resources are relevant to both. Helix [34] and HexGen [24] consider both heterogeneous and geo-distributed resources for LLM inference. Helix uses a time-consuming MILP algorithm for model placement, which is not suitable for environments with high resource availability. HexGen uses a more lightweight dynamic programming approach for splitting tensor and pipeline parallelism across resources, making it more appropriate for dynamic settings. However, both of these works optimize only for performance, ignoring the monetary communication costs that arise in geo-distributed settings. SkyServe [33] places replicas of models across zones and regions, but restricts each replica to a single zone and homogeneous resources. As replicas do not communicate in inference, inter-zone communication is not considered. In contrast, Sailor targets geo-distributed training, which requires frequent communication among data-parallel replicas, substantially increasing scheduling complexity.

8 Conclusion

We propose Sailor, a system for efficient large-scale training over heterogeneous resources with dynamic availability. Sailor co-optimizes the resource allocation and parallelization plan for a training job to optimize a user-defined objective, under constraints. By combining accurate iteration time and memory estimation, dynamic-programming based search, and domain-specific heuristics, Sailor efficiently navigates the large search space of possible job configurations. Sailor’s distributed training framework supports heterogeneous setups and provides seamless elasticity. Sailor achieves 1.1-5.9× higher throughput than baselines across homogeneous, heterogeneous, and geo-distributed settings.

9 Acknowledgements

We thank the SOSP’25 anonymous reviewers and our shepherd, Ionel Gog, for their insightful feedback. We also thank Christina Giannoula for her feedback on the paper, and Michal Friedman for the helpful discussions. This work was supported under project ID infra02 as part of the Swiss AI Initiative, through a grant from the ETH Domain and computational resources provided by the Swiss National Supercomputing Centre (CSCS) under the Alps infrastructure. We thank the CSCS team for their technical support. Foteini Strati is supported by the Swiss National Science Foundation (project number 200021_204620). George Manos is an Onassis Foundation scholar.

References

- [1] Bilge Acun, Benjamin Lee, Fiodar Kazhamiaka, Kiwan Maeng, Udit Gupta, Manoj Chakkaravarthy, David Brooks, and Carole-Jean Wu. 2023. Carbon Explorer: A Holistic Framework for Designing Carbon Aware Datacenters. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Vancouver, BC, Canada) (ASPLOS 2023). Association for Computing Machinery, New York, NY, USA, 118–132. <https://doi.org/10.1145/3575693.3575754>
- [2] Omid Alipourfard, Hongqiang Harry Liu, Jianshu Chen, Shivaram Venkataraman, Minlan Yu, and Ming Zhang. 2017. CherryPick: Adaptively Unearthing the Best Cloud Configurations for Big Data Analytics. In *14th USENIX Symposium on Networked Systems Design and Implementation (NSDI 17)*. USENIX Association, Boston, MA, 469–482. <https://www.usenix.org/conference/nsdi17/technical-sessions/presentation/alipourfard>
- [3] Sanjith Athlur, Nitika Saran, Muthian Sivathanu, Ramachandran Ramjee, and Nipun Kwatra. 2022. Varuna: scalable, low-cost training of massive deep learning models. In *Proceedings of the Seventeenth European Conference on Computer Systems* (Rennes, France) (EuroSys '22). Association for Computing Machinery, New York, NY, USA, 472–487. <https://doi.org/10.1145/3492321.3519584>
- [4] AWS. 2025. Overview of Data Transfer Costs for Common Architectures. <https://aws.amazon.com/blogs/architecture/overview-of-data-transfer-costs-for-common-architectures/>.
- [5] Azure. 2025. Azure, Bandwidth pricing. <https://azure.microsoft.com/en-us/pricing/details/bandwidth/>.
- [6] Baseline. 2024. China achieves breakthrough in AI training. <https://www.baselinemag.com/news/china-achieves-breakthrough-in-ai-training/>.
- [7] Tiancheng Chen, Ales Kubicek, Langwen Huang, and Torsten Hoefer. 2025. CrossPipe: Towards Optimal Pipeline Schedules for Cross-Datacenter Training. arXiv:2507.00217 [cs.DC] <https://arxiv.org/abs/2507.00217>
- [8] Runxiang Cheng, Chris Cai, Selman Yilmaz, Rahul Mitra, Malay Bag, Mrinmoy Ghosh, and Tianyin Xu. 2023. Towards GPU Memory Efficiency for Distributed Training at Scale. In *Proceedings of the 2023 ACM Symposium on Cloud Computing* (Santa Cruz, CA, USA) (SoCC '23). Association for Computing Machinery, New York, NY, USA, 281–297. <https://doi.org/10.1145/3620678.3624661>
- [9] Tapan Chugh, Srikanth Kandula, Arvind Krishnamurthy, Ratul Mahajan, and Ishai Menache. 2023. Anticipatory Resource Allocation for ML Training. In *Proceedings of the 2023 ACM Symposium on Cloud Computing* (Santa Cruz, CA, USA) (SoCC '23). Association for Computing Machinery, New York, NY, USA, 410–426. <https://doi.org/10.1145/3620678.3624669>
- [10] Google Cloud. 2024. Google Cloud - About GPUs. <https://cloud.google.com/compute/docs/gpus/about-gpus>.
- [11] Google Cloud. 2025. Google Cloud, All networking pricing. https://pytorch.org/docs/stable/torch_cuda_memory.html.
- [12] deepspeedai. 2025. Megatron-DeepSpeed. <https://github.com/deepspeedai/Megatron-DeepSpeed>.
- [13] Arthur Douillard, Qixuan Feng, Andrei A. Rusu, Rachita Chhaparia, Yani Donchev, Adhiguna Kuncoro, Marc'Aurelio Ranzato, Arthur Szlam, and Jiahun Shen. 2024. DiLoCo: Distributed Low-Communication Training of Language Models. arXiv:2311.08105 [cs.LG] <https://arxiv.org/abs/2311.08105>
- [14] Jiangfei Duan, Ziang Song, Xupeng Miao, Xiaoli Xi, Dahua Lin, Harry Xu, Minjia Zhang, and Zhihao Jia. 2024. Parcae: Proactive, Liveput-Optimized DNN Training on Preemptible Instances. In *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI 24)*. USENIX Association, Santa Clara, CA, 1121–1139. <https://www.usenix.org/conference/nsdi24/presentation/duan>
- [15] Juncheng Gu, Mosharaf Chowdhury, Kang G. Shin, Yibo Zhu, Myeongjae Jeon, Junjie Qian, Hongqiang Liu, and Chuanxiong Guo. 2019. Tiresias: A GPU Cluster Manager for Distributed Deep Learning. In *16th USENIX Symposium on Networked Systems Design and Implementation (NSDI 19)*. USENIX Association, Boston, MA, 485–500. <https://www.usenix.org/conference/nsdi19/presentation/gu>
- [16] Runsheng Benson Guo, Utkarsh Anand, Arthur Chen, and Khuzaima Daudjee. 2024. Cephalo: Harnessing Heterogeneous GPU Clusters for Training Transformer Models. arXiv:2411.01075 [cs.DC] <https://arxiv.org/abs/2411.01075>
- [17] Toms Hardware. 2025. Meta to build 2GW data center with over 1.3 million Nvidia AI GPUs.
- [18] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Mia Xu Chen, Dehao Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V. Le, Yonghui Wu, and Zhifeng Chen. 2019. GPipe: efficient training of giant neural networks using pipeline parallelism. Curran Associates Inc., Red Hook, NY, USA.
- [19] HuggingFace. 2025. HuggingFace, GPT-Neo. https://huggingface.co/docs/transformers/en/model_doc/gpt_neo.
- [20] HuggingFace. 2025. HuggingFace, OPT. https://huggingface.co/docs/transformers/en/model_doc/opt.
- [21] Insu Jang, Zhenning Yang, Zhen Zhang, Xin Jin, and Mosharaf Chowdhury. 2023. Ooblock: Resilient Distributed Training of Large Models Using Pipeline Templates. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. ACM. <https://doi.org/10.1145/3600006.3613152>
- [22] Myeongjae Jeon, Shivaram Venkataraman, Amar Phanishayee, Junjie Qian, Wencong Xiao, and Fan Yang. 2019. Analysis of Large-Scale Multi-Tenant GPU Clusters for DNN Training Workloads. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*. USENIX Association, Renton, WA, 947–960. <https://www.usenix.org/conference/atc19/presentation/jeon>
- [23] Xianyan Jia, Le Jiang, Ang Wang, Wencong Xiao, Ziji Shi, Jie Zhang, Xinyuan Li, Langshi Chen, Yong Li, Zhen Zheng, Xiaoyong Liu, and Wei Lin. 2022. Whale: Efficient Giant Model Training over Heterogeneous GPUs. In *2022 USENIX Annual Technical Conference (USENIX ATC 22)*. USENIX Association, Carlsbad, CA, 673–688. <https://www.usenix.org/conference/atc22/presentation/jia-xianyan>
- [24] Youhe Jiang, Ran Yan, Xiaozhe Yao, Yang Zhou, Beidi Chen, and Binhang Yuan. 2024. HEXGEN: generative inference of large language model over heterogeneous environment. In *Proceedings of the 41st International Conference on Machine Learning* (Vienna, Austria) (ICML '24). JMLR.org, Article 881, 16 pages.
- [25] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, Yulu Jia, Sun He, Hongmin Chen, Zhihao Bai, Qi Hou, Shipeng Yan, Ding Zhou, Yiyao Sheng, Zhuo Jiang, Haohan Xu, Haoran Wei, Zhang Zhang, Pengfei Nie, Leqi Zou, Sida Zhao, Liang Xiang, Zherui Liu, Zhe Li, Xiaoying Jia, Jianxi Ye, Xin Jin, and Xin Liu. 2024. MegaScale: Scaling Large Language Model Training to More Than 10,000 GPUs. arXiv:2402.15627 [cs.LG]
- [26] Kyungyong Lee and Myungjun Son. 2017. DeepSpotCloud: Leveraging Cross-Region GPU Spot Instances for Deep Learning. In *2017 IEEE 10th International Conference on Cloud Computing (CLOUD)*. 98–105. <https://doi.org/10.1109/CLOUD.2017.21>
- [27] Dacheng Li, Hongyi Wang, Eric Xing, and Hao Zhang. 2022. AMP: Automatically Finding Model Parallel Strategies with Heterogeneity Awareness. arXiv:2210.07297 [cs.LG]
- [28] Qian Li, Bin Li, Pietro Mercati, Ramesh Illikkal, Charlie Tai, Michael Kishinevsky, and Christos Kozyrakis. 2021. RAMBO: Resource Allocation for Microservices Using Bayesian Optimization. *IEEE Computer Architecture Letters* 20, 1 (2021), 46–49. <https://doi.org/10.1109/LCA.2021.3066142>
- [29] Zhiqi Lin, Youshan Miao, Quanlu Zhang, Fan Yang, Yi Zhu, Cheng Li, Saeed Maleki, Xu Cao, Ning Shang, Yilei Yang, Weijiang Xu, Mao Yang, Lintao Zhang, and Lidong Zhou. 2024. nnScaler: Constraint-Guided

- Parallelization Plan Generation for Deep Learning Training. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24)*. USENIX Association, Santa Clara, CA, 347–363. <https://www.usenix.org/conference/osdi24/presentation/lin-zhiqi>
- [30] LinkedIn. 2024. The Heat Challenge of AI Infrastructure: A Growing Concern for Traditional Office Buildings and Older Data Centers. <https://www.linkedin.com/pulse/heat-challenge-ai-infrastructure-gpu-servers-trgdatacenter-b6vcc>.
- [31] Guodong Liu, Youshan Miao, Zhiqi Lin, Xiaoxiang Shi, Saeed Maleki, Fan Yang, Yungang Bao, and Sa Wang. 2024. Aceso: Efficient Parallel DNN Training through Iterative Bottleneck Alleviation. In *Proceedings of the Nineteenth European Conference on Computer Systems (<conf-loc>, <city>Athens</city>, <country>Greece</country>, </conf-loc>)* (EuroSys '24). Association for Computing Machinery, New York, NY, USA, 163–181. <https://doi.org/10.1145/3627703.3629554>
- [32] Liang Luo, Peter West, Pratyush Patel, Arvind Krishnamurthy, and Luis Ceze. 2022. SRIFFY: Swift and Thrifty Distributed Neural Network Training on the Cloud. In *Proceedings of Machine Learning and Systems*, D. Marculescu, Y. Chi, and C. Wu (Eds.), Vol. 4. 833–847. https://proceedings.mlsys.org/paper_files/paper/2022/file/0cafb7890f6a7d4de65507d5bb7e0187-Paper.pdf
- [33] Ziming Mao, Tian Xia, Zhanghao Wu, Wei-Lin Chiang, Tyler Griggs, Romil Bhardwaj, Zongheng Yang, Scott Shenker, and Ion Stoica. 2025. SkyServe: Serving AI Models across Regions and Clouds with Spot Instances. In *Proceedings of the Twentieth European Conference on Computer Systems (Rotterdam, Netherlands) (EuroSys '25)*. Association for Computing Machinery, New York, NY, USA, 159–175. <https://doi.org/10.1145/3689031.3717459>
- [34] Yixuan Mei, Yonghao Zhuang, Xupeng Miao, Juncheng Yang, Zhihao Jia, and Rashmi Vinayak. 2025. Helix: Serving Large Language Models over Heterogeneous GPUs and Network via Max-Flow. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (Rotterdam, Netherlands) (ASPLOS '25)*. Association for Computing Machinery, New York, NY, USA, 586–602. <https://doi.org/10.1145/3669940.3707215>
- [35] Meta. 2025. The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation. <https://ai.meta.com/blog/llama-4-multimodal-intelligence/>.
- [36] Xupeng Miao, Yining Shi, Zhi Yang, Bin Cui, and Zhihao Jia. 2023. SDPipe: A Semi-Decentralized Framework for Heterogeneity-Aware Pipeline-parallel Training. *Proc. VLDB Endow.* 16, 9 (may 2023), 2354–2363. <https://doi.org/10.14778/3598581.3598604>
- [37] Xupeng Miao, Yujie Wang, Youhe Jiang, Chunan Shi, Xiaonan Nie, Hailin Zhang, and Bin Cui. 2022. Galvatron: Efficient Transformer Training over Multiple GPUs Using Automatic Parallelism. *Proc. VLDB Endow.* 16, 3 (2022), 470–479. <https://doi.org/10.14778/3570690.3570697>
- [38] Zizhao Mo, Huanle Xu, and Chengzhong Xu. 2024. Heet: Accelerating Elastic Training in Heterogeneous Deep Learning Clusters. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (La Jolla, CA, USA) (ASPLOS '24)*. Association for Computing Machinery, New York, NY, USA, 499–513. <https://doi.org/10.1145/3620665.3640375>
- [39] Jayashree Mohan, Amar Phanishayee, and Vijay Chidambaram. 2021. CheckFreq: Frequent, Fine-Grained DNN Checkpointing. In *19th USENIX Conference on File and Storage Technologies (FAST '21)*. USENIX Association, 203–216. <https://www.usenix.org/conference/fast21/presentation/mohan>
- [40] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R. Devanur, Gregory R. Ganger, Phillip B. Gibbons, and Matei Zaharia. 2019. PipeDream: generalized pipeline parallelism for DNN training. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (Huntsville, Ontario, Canada) (SOSP '19)*. Association for Computing Machinery, New York, NY, USA, 1–15. <https://doi.org/10.1145/3341301.3359646>
- [41] Palak, Rohan Gandhi, Karan Tandon, Debopam Bhattacharjee, and Venkata N. Padmanabhan. 2024. Improving training time and GPU utilization in geo-distributed language model training. arXiv:2411.14458 [cs.DC] <https://arxiv.org/abs/2411.14458>
- [42] Palak, Rohan Gandhi, Karan Tandon, Debopam Bhattacharjee, and Venkata N. Padmanabhan. 2024. Improving training time and GPU utilization in geo-distributed language model training. arXiv:2411.14458 [cs.DC] <https://arxiv.org/abs/2411.14458>
- [43] PyTorch. 2025. PyTorch CUDA Events. <https://pytorch.org/docs/stable/generated/torch.cuda.Event.html>.
- [44] PyTorch. 2025. PyTorch hooks. https://pytorch.org/docs/stable/generated/torch.Tensor.register_hook.html.
- [45] PyTorch. 2025. Understanding CUDA Memory Usage. https://pytorch.org/docs/stable/torch_cuda_memory.html.
- [46] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. ZeRO-infinity: breaking the GPU memory wall for extreme scale deep learning. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (<conf-loc>, <city>St. Louis</city>, <state>Missouri</state>, </conf-loc>)* (SC '21). Association for Computing Machinery, New York, NY, USA, Article 59, 14 pages. <https://doi.org/10.1145/3458817.3476205>
- [47] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. ZeRO-Offload: Democratizing Billion-Scale Model Training. In *2021 USENIX Annual Technical Conference (USENIX ATC '21)*. USENIX Association, 551–564. <https://www.usenix.org/conference/atc21/presentation/ren-jie>
- [48] Lisa Rivalin, Lingyun Yi, Megan Diefenbach, Alex Bruefach, Frances Amatrua, and Tobias Tiecke. 2025. Estimating embodied carbon in data center hardware, down to the individual screws. <https://sustainability.atmeta.com/blog/2024/09/10/estimating-embodied-carbon-in-data-center-hardware-down-to-the-individual-screws/>.
- [49] Max Ryabinin, Tim Dettmers, Michael Diskin, and Alexander Borzunov. 2023. SWARM Parallelism: Training Large Models Can Be Surprisingly Communication-Efficient. arXiv:2301.11913 [cs.DC]
- [50] Amedeo Sapio, Marco Canini, Chen-Yu Ho, Jacob Nelson, Panos Kalnis, Changhoon Kim, Arvind Krishnamurthy, Masoud Moshref, Dan Ports, and Peter Richtarik. 2021. Scaling Distributed Machine Learning with In-Network Aggregation. In *18th USENIX Symposium on Networked Systems Design and Implementation (NSDI '21)*. USENIX Association, 785–808. <https://www.usenix.org/conference/nsdi21/presentation/sapio>
- [51] Ian Schneider, Hui Xu, Stephan Benecke, David Patterson, Keguo Huang, Parthasarathy Ranganathan, and Cooper Elsworth. 2025. Life-Cycle Emissions of AI Hardware: A Cradle-To-Grave Approach and Generational Trends. arXiv:2502.01671 [cs.AR] <https://arxiv.org/abs/2502.01671>
- [52] Semaphore. 2024. Microsoft Azure CTO: US data centers will soon hit size limits. <https://www.semaphor.com/article/10/11/2024/microsoft-azure-cto-us-data-centers-will-soon-hit-limits-of-energy-grid>.
- [53] Ruitao Shang, Fei Xu, Zhuoyan Bai, Li Chen, Zhi Zhou, and Fangming Liu. 2023. spotDNN: Provisioning Spot Instances for Predictable Distributed DNN Training in the Cloud. In *2023 IEEE/ACM 31st International Symposium on Quality of Service (IWQoS)*. 1–10. <https://doi.org/10.1109/IWQoS57198.2023.10188717>
- [54] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2020. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism. arXiv:1909.08053 [cs.CL]
- [55] Jovan Stojkovic, Chaojie Zhang, Íñigo Goiri, Esha Choukse, Haoran Qiu, Rodrigo Fonseca, Josep Torrellas, and Ricardo Bianchini. 2025. TAPAS: Thermal- and Power-Aware Scheduling for LLM Inference in

- Cloud Platforms. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2* (Rotterdam, Netherlands) (ASPLOS '25). Association for Computing Machinery, New York, NY, USA, 1266–1281. <https://doi.org/10.1145/3676641.3716025>
- [56] Foteini Strati, Paul Elvinger, Tolga Kerimoglu, and Ana Klimovic. 2024. ML Training with Cloud GPU Shortages: Is Cross-Region the Answer?. In *Proceedings of the 4th Workshop on Machine Learning and Systems* (Athens, Greece) (*EuroMLSys '24*). Association for Computing Machinery, New York, NY, USA, 107–116. <https://doi.org/10.1145/3642970.3655843>
- [57] Foteini Strati, Michal Friedman, and Ana Klimovic. 2025. PCcheck: Persistent Concurrent Checkpointing for ML. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1* (Rotterdam, Netherlands) (ASPLOS '25). Association for Computing Machinery, New York, NY, USA, 811–827. <https://doi.org/10.1145/3669940.3707255>
- [58] Foteini Strati, Sara Mcallister, Amar Phanishayee, Jakub Tarnawski, and Ana Klimovic. 2024. DéjàVu: KV-cache Streaming for Fast, Fault-tolerant Generative LLM Serving. arXiv:2403.01876 [cs.DC] <https://arxiv.org/abs/2403.01876>
- [59] Jakub M Tarnawski, Deepak Narayanan, and Amar Phanishayee. 2021. Piper: Multidimensional Planner for DNN Parallelization. In *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan (Eds.), Vol. 34. Curran Associates, Inc., 24829–24840. https://proceedings.neurips.cc/paper_files/paper/2021/file/d01eeca8b24321cd2fe89dd85b9beb51-Paper.pdf
- [60] John Thorpe, Pengzhan Zhao, Jonathan Eyolfson, Yifan Qiao, Zhihao Jia, Minjia Zhang, Ravi Netravali, and Guoqing Harry Xu. 2023. Bamboo: Making Preemptible Instances Resilient for Affordable Training of Large DNNs. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, Boston, MA, 497–513. <https://www.usenix.org/conference/nsdi23/presentation/thorpe>
- [61] tom's Hardware. 2025. Microsoft surprises analysts with massive 80B AI investment plans for 2025.
- [62] Taegeon Um, Byungsoo Oh, Minyoung Kang, Woo-Yeon Lee, Goeun Kim, Dongseob Kim, Youngtaek Kim, Mohd Muzzammil, and Myeongjae Jeon. 2024. Metis: Fast Automatic Distributed Training on Heterogeneous GPUs. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. USENIX Association, Santa Clara, CA, 563–578. <https://www.usenix.org/conference/atc24/presentation/um>
- [63] Marcel Wagenländer, Guo Li, Bo Zhao, Luo Mai, and Peter Pietzuch. 2024. Tenplex: Dynamic Parallelism for Deep Learning using Parallelizable Tensor Collections. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles* (Austin, TX, USA) (SOSP '24). Association for Computing Machinery, New York, NY, USA, 195–210. <https://doi.org/10.1145/3694715.3695975>
- [64] Jaylen Wang, Daniel S. Berger, Fiodar Kazhamiaka, Celine Irvine, Chaojie Zhang, Esha Choukse, Kali Frost, Rodrigo Fonseca, Brijesh Warriar, Chetan Bansal, Jonathan Stern, Ricardo Bianchini, and Akshitha Sriman. 2024. Designing Cloud Servers for Lower Carbon. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. 452–470. <https://doi.org/10.1109/ISCA59077.2024.00041>
- [65] Jue Wang, Yucheng Lu, Binhang Yuan, Beidi Chen, Percy Liang, Christopher De Sa, Christopher Re, and Ce Zhang. 2023. CocktailSGD: fine-tuning foundation models over 500mbps networks. In *Proceedings of the 40th International Conference on Machine Learning* (Honolulu, Hawaii, USA) (ICML '23). JMLR.org, Article 1497, 19 pages.
- [66] Qizhen Weng, Wencong Xiao, Yinghao Yu, Wei Wang, Cheng Wang, Jian He, Yong Li, Liping Zhang, Wei Lin, and Yu Ding. 2022. MLaaS in the Wild: Workload Analysis and Scheduling in Large-Scale Heterogeneous GPU Clusters. In *19th USENIX Symposium on Networked Systems Design and Implementation (NSDI 22)*. USENIX Association, Renton, WA, 945–960. <https://www.usenix.org/conference/nsdi22/presentation/weng>
- [67] Wikipedia. 2025. GPT-2. [https://en.wikipedia.org/wiki/GPT-2](https://en.wikipedia.org/wiki/GPT-4)
- [68] Wikipedia. 2025. GPT-4. <https://en.wikipedia.org/wiki/GPT-4>
- [69] Wikipedia. 2025. List of Nvidia graphics processing units. https://en.wikipedia.org/wiki/List_of_Nvidia_graphics_processing_units
- [70] Tianyuan Wu, Wei Wang, Yinghao Yu, Siran Yang, Wenchao Wu, Qinkai Duan, Guodong Yang, Jiamang Wang, Lin Qu, and Liping Zhang. 2024. FALCON: Pinpointing and Mitigating Stragglers for Large-Scale Hybrid-Parallel Training. arXiv:2410.12588 [cs.DC] <https://arxiv.org/abs/2410.12588>
- [71] Neeraja J. Yadwadkar, Bharath Hariharan, Joseph E. Gonzalez, Burton Smith, and Randy H. Katz. 2017. Selecting the best VM across multiple public clouds: a data-driven performance modeling approach. In *Proceedings of the 2017 Symposium on Cloud Computing* (Santa Clara, California) (SoCC '17). Association for Computing Machinery, New York, NY, USA, 452–465. <https://doi.org/10.1145/3127479.3131614>
- [72] Ran Yan, Youhe Jiang, Wangcheng Tao, Xiaonan Nie, Bin Cui, and Binhang Yuan. 2024. FlashFlex: Accommodating Large Language Model Training over Heterogeneous Environment. arXiv:2409.01143 [cs.DC] <https://arxiv.org/abs/2409.01143>
- [73] Zongheng Yang, Zhanghao Wu, Michael Luo, Wei-Lin Chiang, Romil Bhardwaj, Woosuk Kwon, Siyuan Zhuang, Frank Sifei Luan, Gautam Mittal, Scott Shenker, and Ion Stoica. 2023. SkyPilot: An Intercloud Broker for Sky Computing. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*. USENIX Association, Boston, MA, 437–455. <https://www.usenix.org/conference/nsdi23/presentation/yang-zongheng>
- [74] Binhang Yuan, Yongjun He, Jared Quincy Davis, Tianyi Zhang, Tri Dao, Beidi Chen, Percy Liang, Christopher Re, and Ce Zhang. 2023. Decentralized Training of Foundation Models in Heterogeneous Environments. arXiv:2206.01288 [cs.DC]
- [75] Tailing Yuan, Yuliang Liu, Xucheng Ye, Shenglong Zhang, Jianchao Tan, Bin Chen, Chengru Song, and Di Zhang. 2024. Accelerating the Training of Large Language Models using Efficient Activation Rematerialization and Optimal Hybrid Parallelism. In *2024 USENIX Annual Technical Conference (USENIX ATC 24)*. USENIX Association, Santa Clara, CA, 545–561. <https://www.usenix.org/conference/atc24/presentation/yuan>
- [76] Hao Zhang, Zeyu Zheng, Shizhen Xu, Wei Dai, Qirong Ho, Xiaodan Liang, Zhiting Hu, Jinliang Wei, Pengtao Xie, and Eric P. Xing. 2017. Poseidon: An Efficient Communication Architecture for Distributed Deep Learning on GPU Clusters. In *2017 USENIX Annual Technical Conference (USENIX ATC 17)*. USENIX Association, Santa Clara, CA, 181–193. <https://www.usenix.org/conference/atc17/technical-sessions/presentation/zhang>
- [77] Haoyue Zheng, Fei Xu, Li Chen, Zhi Zhou, and Fangming Liu. 2019. Cynthia: Cost-Efficient Cloud Resource Provisioning for Predictable Distributed Deep Neural Network Training. In *Proceedings of the 48th International Conference on Parallel Processing* (Kyoto, Japan) (ICPP '19). Association for Computing Machinery, New York, NY, USA, Article 86, 11 pages. <https://doi.org/10.1145/3337821.3337873>
- [78] Lianmin Zheng, Zhuohan Li, Hao Zhang, Yonghao Zhuang, Zhifeng Chen, Yanping Huang, Yida Wang, Yuanzhong Xu, Danyang Zhuo, Eric P. Xing, Joseph E. Gonzalez, and Ion Stoica. 2022. Alpa: Automating Inter- and Intra-Operator Parallelism for Distributed Deep Learning. arXiv:2201.12023 [cs.LG]
- [79] Kan Zhu, Yilong Zhao, Liangyu Zhao, Gefei Zuo, Yile Gu, Dedong Xie, Yufei Gao, Qinyu Xu, Tian Tang, Zihao Ye, Keisuke Kamahori, Chien-Yu Lin, Stephanie Wang, Arvind Krishnamurthy, and Baris Kasikci. 2024. NanoFlow: Towards Optimal Large Language Model Serving Throughput. arXiv:2408.12757 [cs.DC] <https://arxiv.org/abs/2408.12757>